

cnn_implementation

May 12, 2026

1 Convolutional Neural Networks implementation using PyTorch

```
[1]: !pip install torch torchvision matplotlib
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cu128)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cu128)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
```

Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)

Requirement already satisfied: nvidia-cusparselt-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)

Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)

Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)

Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)

Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)

Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)

Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)

Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch) (1.5.3)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)

Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.1)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)

1.1 Configuração de Ambiente e Hiperparâmetros:

Este bloco inicializa o ambiente de desenvolvimento, definindo os recursos computacionais e as constantes globais que guiarão o treinamento.

- Seleção de Dispositivo: Configura o `DEVICE` para utilizar GPU (CUDA) se disponível, garantindo a eficiência no processamento de tensores.
- Hiperparâmetros: Define o `BATCH_SIZE` em 64 para equilibrar o uso de memória e a estabilidade do gradiente, e o `LR` (taxa de aprendizado) em $1e-3$ para o otimizador Adam.
- Bibliotecas: Importa as ferramentas fundamentais: `torch` para a lógica de tensores, `torchvision` para acesso aos datasets e modelos pré-treinados, e `matplotlib` para visualização dos resultados.

```
[2]: import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
import matplotlib.pyplot as plt
import numpy as np
from torch.utils.data import DataLoader

DEVICE      = torch.device("cuda" if torch.cuda.is_available() else "cpu")
BATCH_SIZE  = 64
EPOCHS      = 15
LR           = 1e-3
```

1.2 Pipelines de Dados (DataLoaders)

Implementa o carregamento e a preparação dos dados para o MNIST e o CIFAR-10 através da biblioteca `torchvision`.

- MNIST: Como é um dataset simples de dígitos em escala de cinza, utiliza apenas normalização com média (0.1307) e desvio padrão (0.3081) específicos do dataset para estabilizar o treinamento.
- CIFAR-10 (Data Augmentation): Para lidar com a alta variância de imagens naturais, aplica-se `RandomHorizontalFlip` e `RandomCrop`, o que dobra a diversidade efetiva dos dados e torna a rede mais robusta a translações e espelhamentos.
- Normalização RGB: No CIFAR-10, a normalização é feita por canal (R, G, B) para manter as ativações em uma escala compatível com o esperado pelas redes convolucionais.

```
[3]: def get_mnist_loaders():
    """
    MNIST: escala de cinza 28x28, 10 classes de dígitos.
    Normalização com média/desvio do dataset para estabilizar o gradiente.
    Sem augmentation agressiva - MNIST é simples e limpo.
```

```

"""
tf = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
train = torchvision.datasets.MNIST(root="./data", train=True,
↪download=True, transform=tf)
test = torchvision.datasets.MNIST(root="./data", train=False,
↪download=True, transform=tf)
return (DataLoader(train, batch_size=BATCH_SIZE, shuffle=True,
↪num_workers=2),
        DataLoader(test, batch_size=BATCH_SIZE, shuffle=False,
↪num_workers=2))

def get_cifar10_loaders():
    """
    CIFAR-10: RGB 32×32, 10 classes naturais.
    Data Augmentation (flip + crop aleatório):
    - RandomHorizontalFlip: cria invariância a espelhamento horizontal,
      dobra efetivamente o tamanho do dataset sem custo de coleta.
    - RandomCrop(32, padding=4): simula pequenas translações, forçando a
      rede a aprender features robustas à posição do objeto.
    Normalização com estatísticas do CIFAR-10 (média/desvio por canal RGB).
    """
    tf_train = transforms.Compose([
        transforms.RandomHorizontalFlip(),
        transforms.RandomCrop(32, padding=4),
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465),
                              (0.2023, 0.1994, 0.2010))
    ])
    tf_test = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465),
                              (0.2023, 0.1994, 0.2010))
    ])
    train = torchvision.datasets.CIFAR10(root="./data", train=True,
↪download=True, transform=tf_train)
    test = torchvision.datasets.CIFAR10(root="./data", train=False,
↪download=True, transform=tf_test)
    return (DataLoader(train, batch_size=BATCH_SIZE, shuffle=True,
↪num_workers=2),
            DataLoader(test, batch_size=BATCH_SIZE, shuffle=False,
↪num_workers=2))

```

1.3 Arquiteturas das CNNs Customizadas

Define as classes das redes convolucionais adaptadas para a complexidade de cada tarefa.

CNN_MNIST:

- **Estrutura:** Utiliza dois blocos convolucionais seguidos de camadas densas.
- **Regularização:** Aplica Dropout leve (0.25) nas convoluções para evitar co-adaptação e Weight Decay (L2) no otimizador para penalizar pesos excessivos, garantindo que a rede não memorize o fundo limpo do MNIST.

CNN_CIFAR-10:

- **Profundidade:** Três blocos convolucionais com filtros crescentes (32→64→128) para capturar uma hierarquia de features mais rica (bordas, texturas e objetos).
- **Batch Normalization:** Introduzido após cada convolução para combater o internal covariate shift, acelerando a convergência e permitindo taxas de aprendizado mais altas.
- **Dropout (0.3/0.5):** Configuração mais agressiva para lidar com o risco de overfitting inerente à complexidade do CIFAR-10.

```
[4]: class CNN_MNIST(nn.Module):  
    """  
    Arquitetura simples para MNIST (imagens grayscale 28x28).  
  
    Justificativa de regularização:  
  
    • Weight Decay (L2): penaliza pesos grandes ao somar  $\cdot ||w||^2$  à loss.  
      Evita que a rede memorize padrões específicos do treino. Implementado  
      no otimizador (weight_decay=1e-4). Valor conservador pois MNIST é  
      tarefa simples - decay excessivo prejudicaria convergência.  
  
    • Dropout leve (p=0.25 conv / p=0.5 fc): desativa neurônios aleatórios  
      durante o treino, forçando redundância de representação. Dropout mais  
      pesado nas camadas densas, onde overfitting tende a ser maior.  
      Valor baixo nas convoluções para não destruir mapas de features  
      espaciais compactos (28x28 já é pequeno).  
  
    Fluxo: Conv→ReLU→Pool → Conv→ReLU→Pool → FC → saída (10 classes)  
    """  
    def __init__(self):  
        super().__init__()  
        self.features = nn.Sequential(  
            # Bloco 1: extrai bordas e texturas simples  
            nn.Conv2d(1, 32, kernel_size=3, padding=1), # 28x28→28x28  
            nn.ReLU(),  
            nn.MaxPool2d(2), # 28x28→14x14  
            nn.Dropout2d(0.25),
```

```

        # Bloco 2: combina features de baixo nível em padrões complexos
        nn.Conv2d(32, 64, kernel_size=3, padding=1), # 14×14→14×14
        nn.ReLU(),
        nn.MaxPool2d(2), # 14×14→7×7
        nn.Dropout2d(0.25),
    )
    self.classifier = nn.Sequential(
        nn.Flatten(),
        nn.Linear(64 * 7 * 7, 128),
        nn.ReLU(),
        nn.Dropout(0.5), # Dropout pesado na FC - principal fonte de
↪overfitting
        nn.Linear(128, 10)
    )

    def forward(self, x):
        return self.classifier(self.features(x))

class CNN_CIFAR10(nn.Module):
    """
    Arquitetura mais profunda para CIFAR-10 (RGB 32×32, alta variância
↪intra-classe).

    Justificativas de projeto:

    • Profundidade (3 blocos conv): CIFAR-10 requer hierarquia de features
      mais rica - bordas → texturas → partes → objetos. Redes rasas falham
      em capturar essa hierarquia semântica.

    • Batch Normalization (após cada conv): normaliza as ativações por batch,
      reduzindo o "internal covariate shift". Permite LR maior, acelera
      convergência e atua como regularizador implícito (adiciona ruído de
      estimativa das estatísticas do mini-batch).

    • Dropout moderado (p=0.3-0.5): CIFAR-10 tem 50 k amostras de treino
      e alta variância visual → risco real de overfitting. Dropout 30% nos
      blocos convolucionais e 50% nas FC equilibra capacidade e generalização.

    • Filtros crescentes (32→64→128): canais crescentes capturam
      representações progressivamente mais abstratas e de maior dimensão.

    Fluxo: [Conv→BN→ReLU→Pool→Drop] × 3 → FC → saída (10 classes)
    """
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(

```

```

    # Bloco 1
    nn.Conv2d(3, 32, kernel_size=3, padding=1),
    nn.BatchNorm2d(32),
    nn.ReLU(),
    nn.MaxPool2d(2),          # 32→16
    nn.Dropout2d(0.3),

    # Bloco 2
    nn.Conv2d(32, 64, kernel_size=3, padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU(),
    nn.MaxPool2d(2),          # 16→8
    nn.Dropout2d(0.3),

    # Bloco 3
    nn.Conv2d(64, 128, kernel_size=3, padding=1),
    nn.BatchNorm2d(128),
    nn.ReLU(),
    nn.MaxPool2d(2),          # 8→4
    nn.Dropout2d(0.3),
)
self.classifier = nn.Sequential(
    nn.Flatten(),
    nn.Linear(128 * 4 * 4, 256),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(256, 10)
)

def forward(self, x):
    return self.classifier(self.features(x))

```

1.4 Loop de Treino, Avaliação e Early Stopping

Gerencia o ciclo de vida do treinamento dos modelos.

- **Funções de Apoio:** `train_one_epoch` executa o backpropagation e atualiza os pesos, enquanto `evaluate` calcula a performance no conjunto de teste em modo `eval` (desativando Dropouts).
- **Early Stopping:** Monitora a acurácia de teste. Se não houver melhora real (definida por `min_delta`) durante um período de `patience`, o treinamento é interrompido para evitar o sobreajuste e os melhores pesos são restaurados.
- **Agendamento de LR:** Utiliza um `StepLR` que reduz a taxa de aprendizado pela metade a cada 5 épocas, refinando a busca pelo mínimo da função de perda.

```

[5]: def train_one_epoch(model, loader, optimizer, criterion):
    model.train()

```

```

total_loss, correct, total = 0.0, 0, 0
for imgs, labels in loader:
    imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
    optimizer.zero_grad()
    out = model(imgs)
    loss = criterion(out, labels)
    loss.backward()
    optimizer.step()
    total_loss += loss.item() * imgs.size(0)
    correct    += out.argmax(1).eq(labels).sum().item()
    total      += imgs.size(0)
return total_loss / total, correct / total

@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    for imgs, labels in loader:
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        out = model(imgs)
        loss = criterion(out, labels)
        total_loss += loss.item() * imgs.size(0)
        correct    += out.argmax(1).eq(labels).sum().item()
        total      += imgs.size(0)
    return total_loss / total, correct / total

def train_model(model, train_loader, test_loader, tag="modelo",
                weight_decay=1e-4, epochs=EPOCHS,
                patience=3, min_delta=1e-4):          # <-- parâmetros novos
    """
    Early Stopping: interrompe o treino quando a acurácia de teste para de
    melhorar. 'patience' define quantas épocas sem melhora são toleradas;
    'min_delta' é o ganho mínimo considerado melhora real (evita parar por
    flutuações numéricas insignificantes, como 99.24% → 99.28%).
    """
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=LR, weight_decay=weight_decay)
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)

    history = {"train_loss": [], "test_loss": [], "train_acc": [], "test_acc": []}

    best_acc      = 0.0
    epochs_no_imp = 0
    best_state    = None

```



```

    for epoch in range(1, epochs + 1):
        tr_loss, tr_acc = train_one_epoch(model, train_loader, optimizer,
↪criterion)
        te_loss, te_acc = evaluate(model, test_loader, criterion)
        scheduler.step()

        history["train_loss"].append(tr_loss)
        history["test_loss"].append(te_loss)
        history["train_acc"].append(tr_acc)
        history["test_acc"].append(te_acc)

        print(f"[{tag}] Época {epoch:02d}/{epochs} | "
              f"Loss Treino: {tr_loss:.4f} | Acc Treino: {tr_acc*100:.2f}% | "
              f"Loss Teste: {te_loss:.4f} | Acc Teste: {te_acc*100:.2f}%")

        # Early Stopping
        if te_acc > best_acc + min_delta:
            best_acc = te_acc
            epochs_no_imp = 0
            best_state = {k: v.clone() for k, v in model.state_dict().
↪items()}
            print(f"    Nova melhor acurácia: {best_acc*100:.2f}% - pesos_
↪salvos.")
        else:
            epochs_no_imp += 1
            print(f"    Sem melhora ({epochs_no_imp}/{patience}). Melhor:
↪{best_acc*100:.2f}%")
            if epochs_no_imp >= patience:
                print(f"\n Early stopping na época {epoch}. Restaurando_
↪melhores pesos.")
                model.load_state_dict(best_state)
                break

        #

    return history

```

1.5 Visualização de Desempenho e Comparação

Ferramentas gráficas para cumprir a exigência de análise visual da atividade.

- `plot_history`: Gera gráficos de linha comparando as curvas de perda (Loss) e acurácia entre os conjuntos de treino e teste ao longo das épocas.
- `plot_comparison`: Cria um gráfico de barras para a comparação final entre todos os modelos testados (Custom vs. ResNet), permitindo identificar rapidamente qual arquitetura obteve a melhor performance preditiva.

```
[6]: def plot_history(history, title=""):
    """Plota curvas de loss e acurácia lado a lado."""
    epochs = range(1, len(history["train_loss"]) + 1)
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    fig.suptitle(title, fontsize=14, fontweight="bold")

    # Loss
    axes[0].plot(epochs, history["train_loss"], "b-o", label="Treino", ↵
    ↵markersize=4)
    axes[0].plot(epochs, history["test_loss"], "r-o", label="Teste", ↵
    ↵markersize=4)
    axes[0].set_title("Cross-Entropy Loss")
    axes[0].set_xlabel("Época"); axes[0].set_ylabel("Loss")
    axes[0].legend(); axes[0].grid(alpha=0.3)

    # Acurácia
    axes[1].plot(epochs, [a * 100 for a in history["train_acc"]], "b-o", ↵
    ↵label="Treino", markersize=4)
    axes[1].plot(epochs, [a * 100 for a in history["test_acc"]], "r-o", ↵
    ↵label="Teste", markersize=4)
    axes[1].set_title("Acurácia (%)")
    axes[1].set_xlabel("Época"); axes[1].set_ylabel("Acurácia (%)")
    axes[1].legend(); axes[1].grid(alpha=0.3)

    plt.tight_layout()
    plt.show()

def plot_comparison(results: dict, metric="test_acc"):
    """Gráfico de barras comparando modelos."""
    names = list(results.keys())
    values = [max(h[metric]) * 100 for h in results.values()]
    colors = plt.cm.Set2(np.linspace(0, 1, len(names)))

    fig, ax = plt.subplots(figsize=(9, 5))
    bars = ax.bar(names, values, color=colors, edgecolor="white", linewidth=1.2)
    ax.bar_label(bars, fmt="%.2f%", padding=4, fontsize=11, fontweight="bold")
    ax.set_ylabel("Acurácia no Teste (%)", fontsize=12)
    ax.set_title(f"Comparação de Modelos - {metric.replace('_', ' ').title()}", ↵
    ↵fontsize=13)
    ax.set_ylim(0, 105)
    ax.grid(axis="y", alpha=0.3)
    plt.tight_layout()
    plt.show()
```

1.6 Transfer Learning com ResNet50

Adapta uma arquitetura estado-da-arte pré-treinada para os datasets da atividade.

- **Adaptação Grayscale:** Para o MNIST, a primeira camada convolucional da ResNet é substituída para aceitar 1 canal em vez de 3.
- **Fine-tuning:** O classificador final (fc) é trocado para refletir as 10 classes do problema. Como a ResNet foi treinada no ImageNet, ela já possui detectores de formas e texturas universais, exigindo poucas épocas de ajuste fino para superar modelos customizados.
- **Redimensionamento:** As imagens são redimensionadas para 224×224, o padrão esperado pela arquitetura original da ResNet50.

```
[7]: def build_resnet50_for(num_classes=10, grayscale=False):  
    """  
    Carrega ResNet-50 pré-treinada no ImageNet e adapta para o dataset alvo.  
  
    Transfer Learning:  
  
    • Pesos ImageNet já codificam features ricas (bordas, texturas, formas).  
      Fine-tuning converge mais rápido e generaliza melhor com poucos dados.  
    • Substituímos apenas o classificador final (fc) para o número de classes  
      desejado, preservando o backbone convolucional.  
    • Para MNIST (1 canal): substituímos a primeira conv por uma equivalente  
      de 1 canal para aceitar imagens grayscale.  
    • Redimensionamento para 224×224 obrigatório - ResNet foi projetada para  
      esse tamanho; entradas menores degradam as features dos primeiros blocos.  
    """  
    model = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)  
  
    if grayscale:  
        # Adapta o primeiro conv para 1 canal (MNIST)  
        model.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3,   
↪ bias=False)  
  
        # Substitui o classificador final  
        in_features = model.fc.in_features  
        model.fc = nn.Linear(in_features, num_classes)  
  
    return model.to(DEVICE)  
  
def get_resnet_loaders_mnist():  
    """MNIST redimensionado para 224×224 (exigência da ResNet)."""  
    tf = transforms.Compose([  
        transforms.Resize(224),  
        transforms.ToTensor(),  
        transforms.Normalize((0.1307,), (0.3081,))  
    ])
```

```

    ])
    train = torchvision.datasets.MNIST(root="./data", train=True,
↪download=True, transform=tf)
    test = torchvision.datasets.MNIST(root="./data", train=False,
↪download=True, transform=tf)
    return (DataLoader(train, batch_size=32, shuffle=True, num_workers=2),
            DataLoader(test, batch_size=32, shuffle=False, num_workers=2))

def get_resnet_loaders_cifar10():
    """CIFAR-10 redimensionado para 224x224."""
    tf_train = transforms.Compose([
        transforms.Resize(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465),
                              (0.2023, 0.1994, 0.2010))
    ])
    tf_test = transforms.Compose([
        transforms.Resize(224),
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465),
                              (0.2023, 0.1994, 0.2010))
    ])
    train = torchvision.datasets.CIFAR10(root="./data", train=True,
↪download=True, transform=tf_train)
    test = torchvision.datasets.CIFAR10(root="./data", train=False,
↪download=True, transform=tf_test)
    return (DataLoader(train, batch_size=32, shuffle=True, num_workers=2),
            DataLoader(test, batch_size=32, shuffle=False, num_workers=2))

```

1.7 Desafio — Visualização de Ativações e Kernels

Implementa o desafio técnico proposto para entender a “caixa preta” da rede.

- **Hooks de Forward:** Utiliza hooks para interceptar a saída da primeira camada convolucional e plotar os mapas de ativação, visualizando quais partes da imagem ativam cada neurônio.
- **Visualização de Pesos:** Plota os pesos brutos dos filtros (kernels), permitindo ver os detectores de bordas e gradientes aprendidos pela rede durante o treinamento.

```

[8]: def plot_kernel_activations(model, sample_input, layer_name="primeira conv"):
    """
    Captura e plota os mapas de ativação da primeira camada convolucional.

    Os kernels da 1ª conv aprendem detectores de baixo nível:
    bordas, gradientes de cor, texturas e orientações.

```

```

Visualizá-los ajuda a depurar e a entender o que a rede aprendeu.
"""
model.eval()

# Obtém o primeiro módulo Conv2d
first_conv = None
for m in model.modules():
    if isinstance(m, nn.Conv2d):
        first_conv = m
        break

if first_conv is None:
    print("Nenhuma camada Conv2d encontrada.")
    return

# Hook para capturar saída da camada
activations = {}
def hook_fn(module, input, output):
    activations["out"] = output.detach()

handle = first_conv.register_forward_hook(hook_fn)

with torch.no_grad():
    inp = sample_input.unsqueeze(0).to(DEVICE)
    model(inp)

handle.remove()

act = activations["out"].squeeze(0).cpu()    # (C_out, H, W)
n_maps = min(act.shape[0], 32)              # Exibe até 32 mapas

cols = 8
rows = (n_maps + cols - 1) // cols
fig, axes = plt.subplots(rows, cols, figsize=(cols * 2, rows * 2))
fig.suptitle(f"Ativações da {layer_name} - {n_maps} mapas", fontsize=13)

for i in range(rows * cols):
    ax = axes[i // cols][i % cols] if rows > 1 else axes[i % cols]
    if i < n_maps:
        ax.imshow(act[i].numpy(), cmap="viridis")
        ax.set_title(f"K{i}", fontsize=8)
        ax.axis("off")

plt.tight_layout()
plt.show()

```

```

def plot_conv_weights(model, layer_name="Pesos dos Kernels"):
    """
    Visualiza os pesos (filtros) da primeira camada convolucional.
    Para modelos com entrada RGB, plota os 3 canais sobrepostos.
    """
    first_conv = None
    for m in model.modules():
        if isinstance(m, nn.Conv2d):
            first_conv = m
            break
    if first_conv is None:
        return

    weights = first_conv.weight.data.cpu() # (C_out, C_in, kH, kW)
    n_filters = min(weights.shape[0], 32)

    # Normaliza para [0,1]
    w_min, w_max = weights.min(), weights.max()
    weights = (weights - w_min) / (w_max - w_min + 1e-8)

    cols = 8
    rows = (n_filters + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(cols * 2, rows * 2))
    fig.suptitle(layer_name, fontsize=13)

    for i in range(rows * cols):
        ax = axes[i // cols][i % cols] if rows > 1 else axes[i % cols]
        if i < n_filters:
            f = weights[i] # (C_in, kH, kW)
            if f.shape[0] == 1:
                ax.imshow(f[0].numpy(), cmap="gray")
            elif f.shape[0] == 3:
                ax.imshow(f.permute(1, 2, 0).numpy())
            else:
                ax.imshow(f[0].numpy(), cmap="gray")
            ax.set_title(f"F{i}", fontsize=8)
        ax.axis("off")

    plt.tight_layout()
    plt.show()

```

1.8 Execução dos Experimentos (Main)

Orquestra o fluxo completo do trabalho.

- Realiza o treinamento e avaliação sequencial das quatro configurações: CNN Custom no MNIST, CNN Custom no CIFAR-10, ResNet50 no MNIST e ResNet50 no CIFAR-10.
- Gera os logs de progresso e invoca todas as funções de plotagem para compilar os resultados

finais que serão apresentados no documento de comparação.

```
[10]: if __name__ == "__main__":

    all_results = {}    # Armazena históricos para comparação final

    # 7.1 CNN CUSTOM - MNIST
    print("\n" + "="*65)
    print(" EXPERIMENTO 1: CNN Custom - MNIST")
    print("="*65)

    mnist_train, mnist_test = get_mnist_loaders()
    cnn_mnist = CNN_MNIST().to(DEVICE)
    print(cnn_mnist)

    hist_mnist = train_model(cnn_mnist, mnist_train, mnist_test,
                             tag="CNN-MNIST", weight_decay=1e-4)
    plot_history(hist_mnist, "CNN Custom - MNIST")
    all_results["CNN-MNIST"] = hist_mnist

    # Visualizações do desafio
    sample_mnist, _ = next(iter(mnist_test))
    plot_kernel_activations(cnn_mnist, sample_mnist[0], "1ª Conv (MNIST)")
    plot_conv_weights(cnn_mnist, "Pesos dos Filtros - CNN MNIST")

    # 7.2 CNN CUSTOM - CIFAR-10
    print("\n" + "="*65)
    print(" EXPERIMENTO 2: CNN Custom - CIFAR-10")
    print("="*65)

    cifar_train, cifar_test = get_cifar10_loaders()
    cnn_cifar = CNN_CIFAR10().to(DEVICE)
    print(cnn_cifar)

    hist_cifar = train_model(cnn_cifar, cifar_train, cifar_test,
                             tag="CNN-CIFAR10", weight_decay=1e-4)
    plot_history(hist_cifar, "CNN Custom - CIFAR-10")
    all_results["CNN-CIFAR10"] = hist_cifar

    sample_cifar, _ = next(iter(cifar_test))
    plot_kernel_activations(cnn_cifar, sample_cifar[0], "1ª Conv (CIFAR-10)")
    plot_conv_weights(cnn_cifar, "Pesos dos Filtros - CNN CIFAR-10")

    # 7.3 BASELINE - ResNet50 no MNIST
    print("\n" + "="*65)
```

```

print("  EXPERIMENTO 3: ResNet50 (fine-tuning) - MNIST")
print("="*65)

res_mnist_train, res_mnist_test = get_resnet_loaders_mnist()
resnet_mnist = build_resnet50_for(num_classes=10, grayscale=True)

# Fine-tuning com 5 épocas (pesos pré-treinados já são ricos)
hist_res_mnist = train_model(resnet_mnist, res_mnist_train, res_mnist_test,
                             tag="ResNet50-MNIST", weight_decay=1e-4,
↪epochs=5)
plot_history(hist_res_mnist, "ResNet50 Fine-tuning - MNIST")
all_results["ResNet50-MNIST"] = hist_res_mnist

# 7.4 BASELINE - ResNet50 no CIFAR-10
print("\n" + "="*65)
print("  EXPERIMENTO 4: ResNet50 (fine-tuning) - CIFAR-10")
print("="*65)

res_cifar_train, res_cifar_test = get_resnet_loaders_cifar10()
resnet_cifar = build_resnet50_for(num_classes=10, grayscale=False)

hist_res_cifar = train_model(resnet_cifar, res_cifar_train, res_cifar_test,
                             tag="ResNet50-CIFAR10", weight_decay=1e-4,
↪epochs=5)
plot_history(hist_res_cifar, "ResNet50 Fine-tuning - CIFAR-10")
all_results["ResNet50-CIFAR10"] = hist_res_cifar

# 7.5 GRÁFICO DE COMPARAÇÃO FINAL
print("\n" + "="*65)
print("  COMPARAÇÃO FINAL DE MODELOS")
print("="*65)

plot_comparison(all_results, metric="test_acc")

# Tabela textual de resultados
print(f"\n{'Modelo':<20} {'Melhor Acc. Teste':>18}")
print("-" * 40)
for name, hist in all_results.items():
    best_acc = max(hist["test_acc"]) * 100
    print(f"{name:<20} {best_acc:>17.2f}%")

```

```

=====
EXPERIMENTO 1: CNN Custom - MNIST
=====

```



```

CNN_MNIST(
    (features): Sequential(
      (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
      (3): Dropout2d(p=0.25, inplace=False)
      (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (5): ReLU()
      (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
      (7): Dropout2d(p=0.25, inplace=False)
    )
    (classifier): Sequential(
      (0): Flatten(start_dim=1, end_dim=-1)
      (1): Linear(in_features=3136, out_features=128, bias=True)
      (2): ReLU()
      (3): Dropout(p=0.5, inplace=False)
      (4): Linear(in_features=128, out_features=10, bias=True)
    )
  )
  [CNN-MNIST] Época 01/15 | Loss Treino: 0.2501 | Acc Treino: 92.30% | Loss Teste:
0.0558 | Acc Teste: 98.13%
    Nova melhor acurácia: 98.13% - pesos salvos.
  [CNN-MNIST] Época 02/15 | Loss Treino: 0.1038 | Acc Treino: 97.02% | Loss Teste:
0.0355 | Acc Teste: 98.81%
    Nova melhor acurácia: 98.81% - pesos salvos.
  [CNN-MNIST] Época 03/15 | Loss Treino: 0.0818 | Acc Treino: 97.54% | Loss Teste:
0.0313 | Acc Teste: 99.06%
    Nova melhor acurácia: 99.06% - pesos salvos.
  [CNN-MNIST] Época 04/15 | Loss Treino: 0.0732 | Acc Treino: 97.80% | Loss Teste:
0.0279 | Acc Teste: 99.05%
    Sem melhora (1/3). Melhor: 99.06%
  [CNN-MNIST] Época 05/15 | Loss Treino: 0.0636 | Acc Treino: 98.06% | Loss Teste:
0.0294 | Acc Teste: 99.11%
    Nova melhor acurácia: 99.11% - pesos salvos.
  [CNN-MNIST] Época 06/15 | Loss Treino: 0.0478 | Acc Treino: 98.54% | Loss Teste:
0.0245 | Acc Teste: 99.24%
    Nova melhor acurácia: 99.24% - pesos salvos.
  [CNN-MNIST] Época 07/15 | Loss Treino: 0.0423 | Acc Treino: 98.67% | Loss Teste:
0.0211 | Acc Teste: 99.27%
    Nova melhor acurácia: 99.27% - pesos salvos.
  [CNN-MNIST] Época 08/15 | Loss Treino: 0.0397 | Acc Treino: 98.79% | Loss Teste:
0.0196 | Acc Teste: 99.36%
    Nova melhor acurácia: 99.36% - pesos salvos.
  [CNN-MNIST] Época 09/15 | Loss Treino: 0.0392 | Acc Treino: 98.76% | Loss Teste:
0.0219 | Acc Teste: 99.21%
    Sem melhora (1/3). Melhor: 99.36%

```

[CNN-MNIST] Época 10/15 | Loss Treino: 0.0371 | Acc Treino: 98.84% | Loss Teste: 0.0232 | Acc Teste: 99.25%

Sem melhora (2/3). Melhor: 99.36%

[CNN-MNIST] Época 11/15 | Loss Treino: 0.0302 | Acc Treino: 99.03% | Loss Teste: 0.0187 | Acc Teste: 99.38%

Nova melhor acurácia: 99.38% - pesos salvos.

[CNN-MNIST] Época 12/15 | Loss Treino: 0.0286 | Acc Treino: 99.09% | Loss Teste: 0.0202 | Acc Teste: 99.33%

Sem melhora (1/3). Melhor: 99.38%

[CNN-MNIST] Época 13/15 | Loss Treino: 0.0253 | Acc Treino: 99.17% | Loss Teste: 0.0184 | Acc Teste: 99.46%

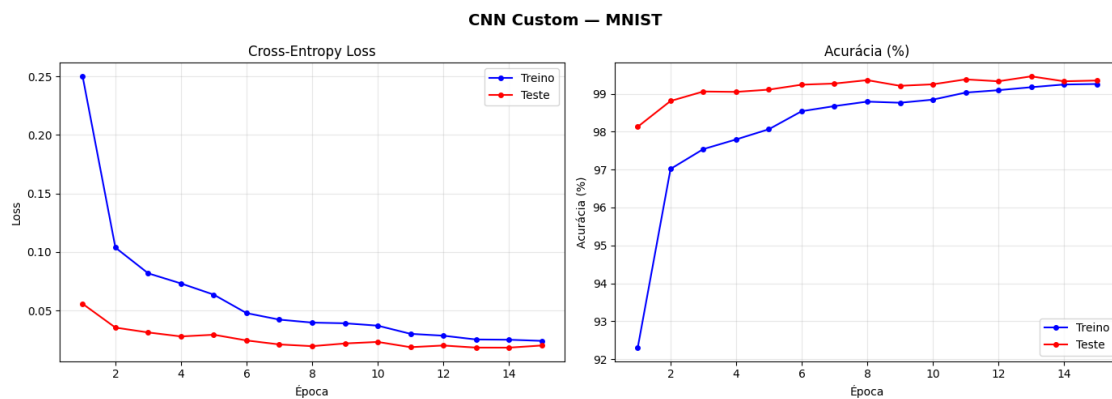
Nova melhor acurácia: 99.46% - pesos salvos.

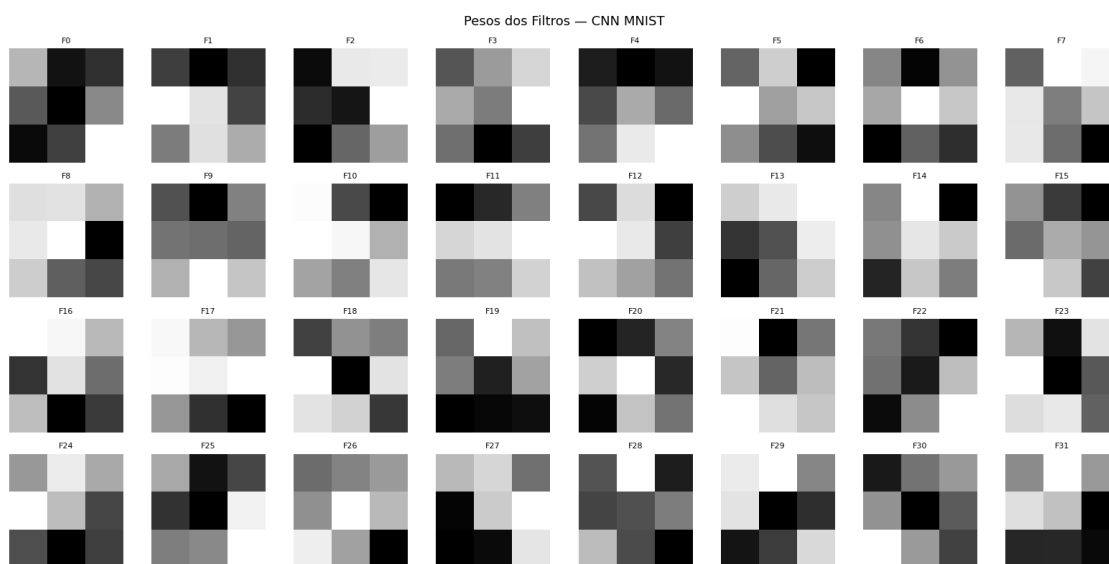
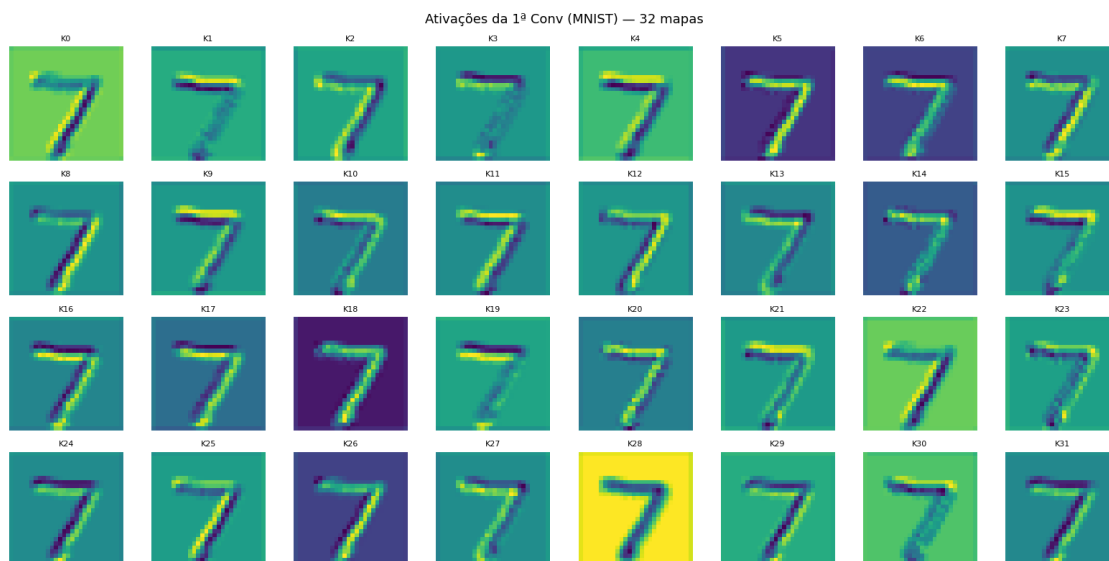
[CNN-MNIST] Época 14/15 | Loss Treino: 0.0251 | Acc Treino: 99.25% | Loss Teste: 0.0183 | Acc Teste: 99.33%

Sem melhora (1/3). Melhor: 99.46%

[CNN-MNIST] Época 15/15 | Loss Treino: 0.0241 | Acc Treino: 99.26% | Loss Teste: 0.0202 | Acc Teste: 99.35%

Sem melhora (2/3). Melhor: 99.46%





```
=====
EXPERIMENTO 2: CNN Custom - CIFAR-10
=====
```

```
100%|      | 170M/170M [00:04<00:00, 37.7MB/s]
```

```
CNN_CIFAR10(
  (features): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
```

```

track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (4): Dropout2d(p=0.3, inplace=False)
    (5): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (6): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (7): ReLU()
    (8): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (9): Dropout2d(p=0.3, inplace=False)
    (10): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (12): ReLU()
    (13): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (14): Dropout2d(p=0.3, inplace=False)
)
(classifier): Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=2048, out_features=256, bias=True)
  (2): ReLU()
  (3): Dropout(p=0.5, inplace=False)
  (4): Linear(in_features=256, out_features=10, bias=True)
)
)
[CNN-CIFAR10] Época 01/15 | Loss Treino: 1.8791 | Acc Treino: 29.68% | Loss
Teste: 1.5072 | Acc Teste: 44.50%
    Nova melhor acurácia: 44.50% - pesos salvos.
[CNN-CIFAR10] Época 02/15 | Loss Treino: 1.7050 | Acc Treino: 37.24% | Loss
Teste: 1.3623 | Acc Teste: 49.69%
    Nova melhor acurácia: 49.69% - pesos salvos.
[CNN-CIFAR10] Época 03/15 | Loss Treino: 1.6210 | Acc Treino: 40.24% | Loss
Teste: 1.2805 | Acc Teste: 52.79%
    Nova melhor acurácia: 52.79% - pesos salvos.
[CNN-CIFAR10] Época 04/15 | Loss Treino: 1.5476 | Acc Treino: 43.35% | Loss
Teste: 1.1975 | Acc Teste: 56.20%
    Nova melhor acurácia: 56.20% - pesos salvos.
[CNN-CIFAR10] Época 05/15 | Loss Treino: 1.4907 | Acc Treino: 45.38% | Loss
Teste: 1.1347 | Acc Teste: 58.25%
    Nova melhor acurácia: 58.25% - pesos salvos.
[CNN-CIFAR10] Época 06/15 | Loss Treino: 1.4142 | Acc Treino: 48.86% | Loss
Teste: 1.0920 | Acc Teste: 60.13%
    Nova melhor acurácia: 60.13% - pesos salvos.
[CNN-CIFAR10] Época 07/15 | Loss Treino: 1.3826 | Acc Treino: 50.14% | Loss
Teste: 1.0461 | Acc Teste: 61.60%

```

Nova melhor acurácia: 61.60% - pesos salvos.
 [CNN-CIFAR10] Época 08/15 | Loss Treino: 1.3441 | Acc Treino: 51.61% | Loss Teste: 1.0183 | Acc Teste: 63.28%

Nova melhor acurácia: 63.28% - pesos salvos.
 [CNN-CIFAR10] Época 09/15 | Loss Treino: 1.3219 | Acc Treino: 52.34% | Loss Teste: 0.9889 | Acc Teste: 64.31%

Nova melhor acurácia: 64.31% - pesos salvos.
 [CNN-CIFAR10] Época 10/15 | Loss Treino: 1.3010 | Acc Treino: 53.58% | Loss Teste: 0.9569 | Acc Teste: 65.49%

Nova melhor acurácia: 65.49% - pesos salvos.
 [CNN-CIFAR10] Época 11/15 | Loss Treino: 1.2564 | Acc Treino: 55.32% | Loss Teste: 0.9303 | Acc Teste: 66.37%

Nova melhor acurácia: 66.37% - pesos salvos.
 [CNN-CIFAR10] Época 12/15 | Loss Treino: 1.2331 | Acc Treino: 55.96% | Loss Teste: 0.9148 | Acc Teste: 67.53%

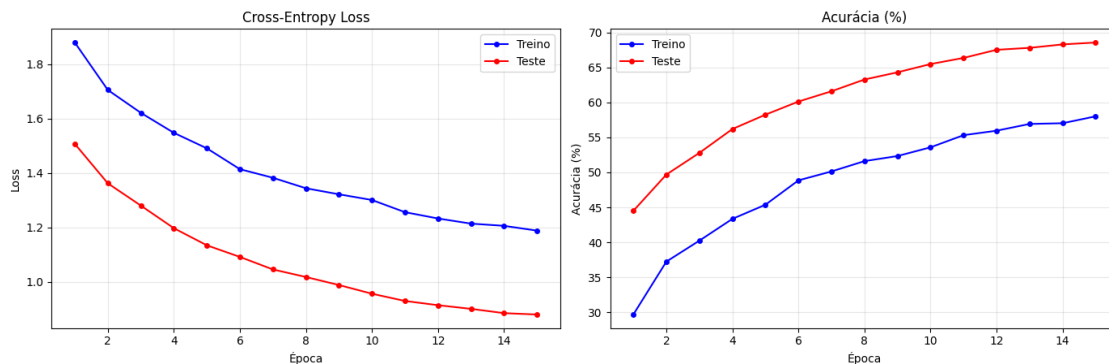
Nova melhor acurácia: 67.53% - pesos salvos.
 [CNN-CIFAR10] Época 13/15 | Loss Treino: 1.2143 | Acc Treino: 56.93% | Loss Teste: 0.9014 | Acc Teste: 67.81%

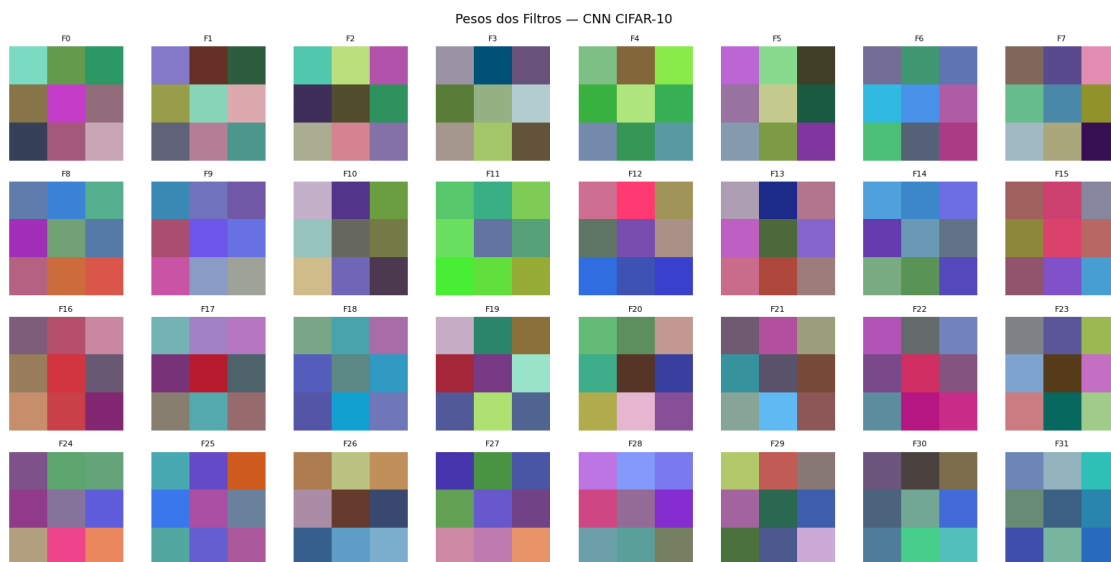
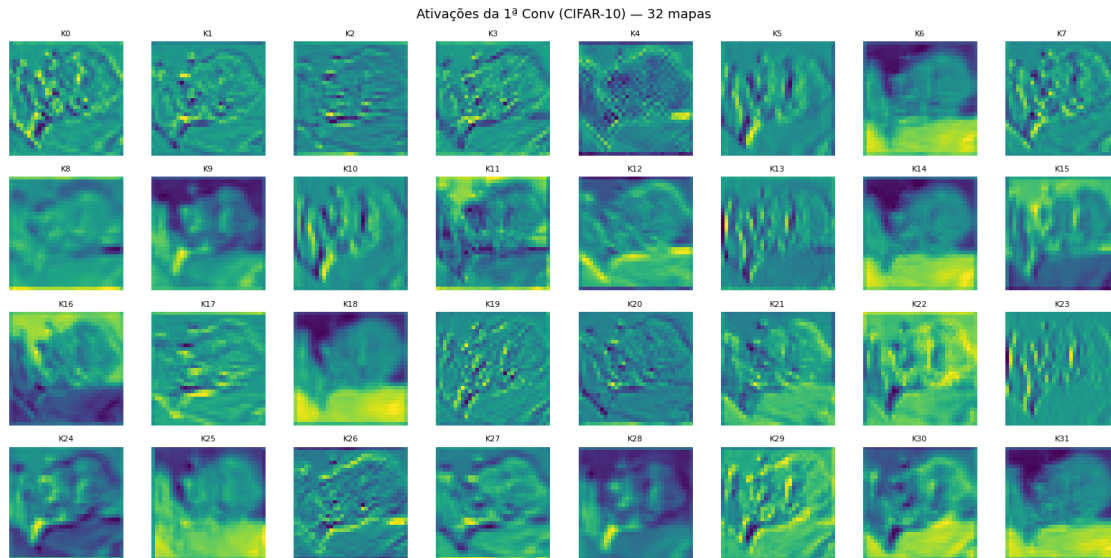
Nova melhor acurácia: 67.81% - pesos salvos.
 [CNN-CIFAR10] Época 14/15 | Loss Treino: 1.2062 | Acc Treino: 57.03% | Loss Teste: 0.8856 | Acc Teste: 68.30%

Nova melhor acurácia: 68.30% - pesos salvos.
 [CNN-CIFAR10] Época 15/15 | Loss Treino: 1.1885 | Acc Treino: 58.01% | Loss Teste: 0.8806 | Acc Teste: 68.57%

Nova melhor acurácia: 68.57% - pesos salvos.

CNN Custom — CIFAR-10





=====

EXPERIMENTO 3: ResNet50 (fine-tuning) - MNIST

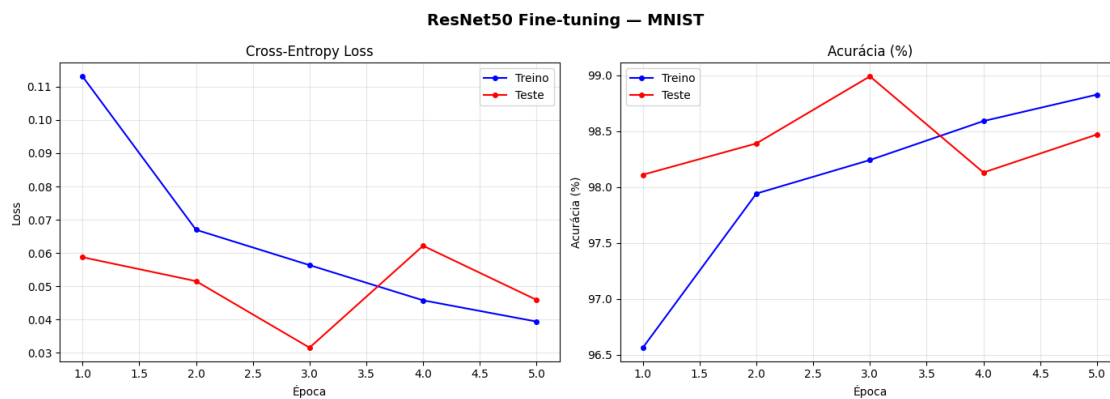
=====

Downloading: "<https://download.pytorch.org/models/resnet50-11ad3fa6.pth>" to
/root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth

100%| | 97.8M/97.8M [00:00<00:00, 187MB/s]

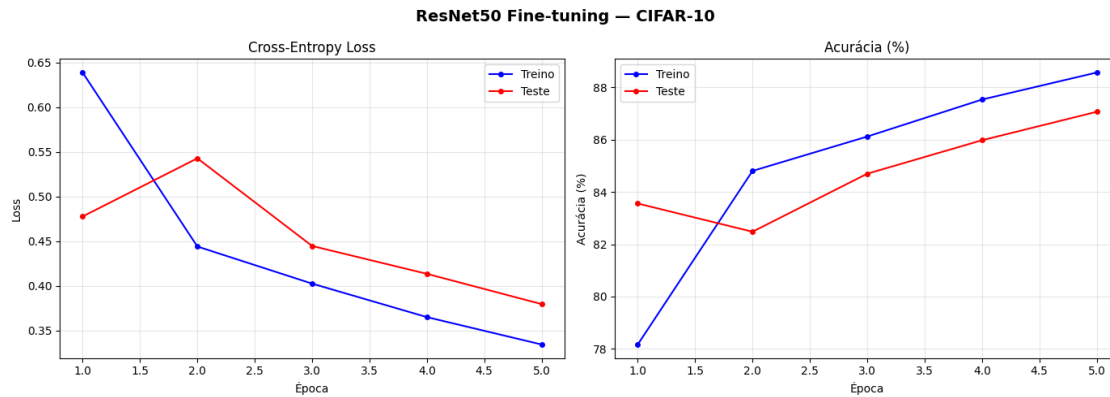
[ResNet50-MNIST] Época 01/5 | Loss Treino: 0.1131 | Acc Treino: 96.56% | Loss
Teste: 0.0588 | Acc Teste: 98.11%

Nova melhor acurácia: 98.11% - pesos salvos.
 [ResNet50-MNIST] Época 02/5 | Loss Treino: 0.0670 | Acc Treino: 97.94% | Loss Teste: 0.0516 | Acc Teste: 98.39%
 Nova melhor acurácia: 98.39% - pesos salvos.
 [ResNet50-MNIST] Época 03/5 | Loss Treino: 0.0564 | Acc Treino: 98.24% | Loss Teste: 0.0316 | Acc Teste: 98.99%
 Nova melhor acurácia: 98.99% - pesos salvos.
 [ResNet50-MNIST] Época 04/5 | Loss Treino: 0.0458 | Acc Treino: 98.59% | Loss Teste: 0.0622 | Acc Teste: 98.13%
 Sem melhora (1/3). Melhor: 98.99%
 [ResNet50-MNIST] Época 05/5 | Loss Treino: 0.0394 | Acc Treino: 98.83% | Loss Teste: 0.0459 | Acc Teste: 98.47%
 Sem melhora (2/3). Melhor: 98.99%



EXPERIMENTO 4: ResNet50 (fine-tuning) - CIFAR-10

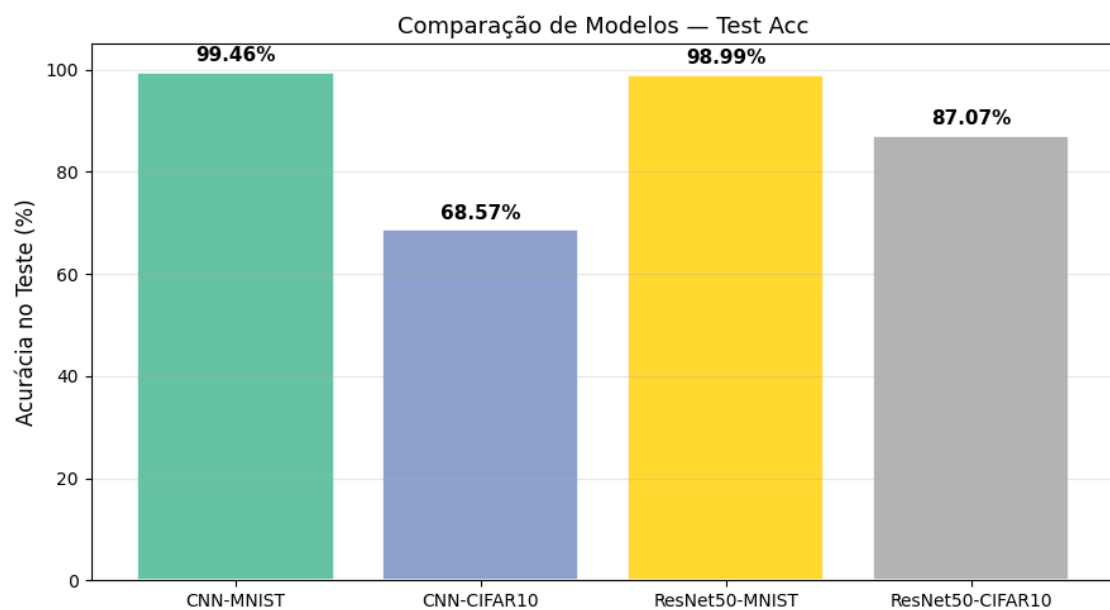
[ResNet50-CIFAR10] Época 01/5 | Loss Treino: 0.6386 | Acc Treino: 78.16% | Loss Teste: 0.4776 | Acc Teste: 83.56%
 Nova melhor acurácia: 83.56% - pesos salvos.
 [ResNet50-CIFAR10] Época 02/5 | Loss Treino: 0.4440 | Acc Treino: 84.81% | Loss Teste: 0.5427 | Acc Teste: 82.48%
 Sem melhora (1/3). Melhor: 83.56%
 [ResNet50-CIFAR10] Época 03/5 | Loss Treino: 0.4024 | Acc Treino: 86.12% | Loss Teste: 0.4446 | Acc Teste: 84.70%
 Nova melhor acurácia: 84.70% - pesos salvos.
 [ResNet50-CIFAR10] Época 04/5 | Loss Treino: 0.3650 | Acc Treino: 87.54% | Loss Teste: 0.4134 | Acc Teste: 85.98%
 Nova melhor acurácia: 85.98% - pesos salvos.
 [ResNet50-CIFAR10] Época 05/5 | Loss Treino: 0.3343 | Acc Treino: 88.57% | Loss Teste: 0.3795 | Acc Teste: 87.07%
 Nova melhor acurácia: 87.07% - pesos salvos.



=====

COMPARAÇÃO FINAL DE MODELOS

=====



Modelo	Melhor Acc. Teste
CNN-MNIST	99.46%
CNN-CIFAR10	68.57%
ResNet50-MNIST	98.99%
ResNet50-CIFAR10	87.07%